This option allows you to set the delivery guarantees of the commit, giving you at-least-once semantics. You can be sure each message is processed before the offset is committed.

The messages you get from this consumer are of type `CommittableMessage`; they are still a `ConsumerRecord`s but with a `CommittableOffset`. In the following snippet, you use the `ConsumerRecord` only to print and then pass the `CommittableOffset` to the next stage of the stream:

```
object ConsumerCommittableSource {
  ...
  Consumer
   .committableSource(consumerSettings, Subscriptions.topics("test2"))    ❶
   .map { msg: CommittableMessage[String, String] =>                      ❶
        println(
          s"${msg.record.key} => ${msg.record.value.toUpperCase}")
        msg.committableOffset}                                            ❷
  ...
}
```

❶ Committable source that produces records of CommittableMessages

❷ Retrieves the offset from the record to pass to the next stage

For simplicity, this example has only one transformation.

Now imagine that you have many messages to process. You probably want to commit in batches; otherwise, the stream will be very slow. To do this, you can use the following `flow`:

```
object ConsumerCommittableSource {
  ...

  Consumer
   ...
   .via(Committer.flow(committerSettings.withMaxBatch(100)))
   ...
}
```